

《数据库系统原理》实验报告（6）					
题目：miniOB 进阶实验					
学号	2452769	姓名	幸可函	日期	2026/4/30
实验环境： 1. 操作系统：Windows 11 2. 容器平台：Docker Desktop for Windows（版本：4.65.0） 3. 数据库镜像：oceanbase/minio:latest					
一. drop-table 1. 赛题基本信息 赛题描述： 本题要求在 MiniOB 中实现删除表功能，即支持执行 drop table 语句，并清除表相关的资源。当前 MiniOB 已经支持建表与创建索引，但缺少删除表功能。因此，在实现 drop table 时，不仅需要删除表在磁盘中的相关文件，还需要释放与该表关联的内存资源，例如已经打开的表对象、索引对象以及缓冲池中的页面等。					
2. 测试用例示例 测试用例一：基础 drop table <pre>create table t(id int, name char(20)); insert into t values(1, 'aaa'); insert into t values(2, 'bbb'); select * from t; drop table t; select * from t;</pre> 预期结果： 前几条语句执行成功，drop table t; 执行成功；删除后再次查询表 t 时，系统返回失败，说明表已经不存在。					
测试用例二：drop 后重建同名表 <pre>create table t2(id int, name char(20)); insert into t2 values(1, 'old'); select * from t2;</pre>					

```
drop table t2;
create table t2(id int, name char(20));
select * from t2;
drop table t2;
```

预期结果：

删除表后可以重新创建同名表，且新建表中不再出现旧数据，说明旧表相关的数据文件和内存缓存已经被正确清除。

测试用例三：删除带索引的表

```
create table t3(id int, name char(20));
insert into t3 values(1, 'aaa');
insert into t3 values(2, 'bbb');
create index idx_t3_id on t3(id);
select * from t3;
drop table t3;
create table t3(id int, name char(20));
select * from t3;
drop table t3;
```

预期结果：

带索引的表可以被成功删除；删除后重新创建同名表并查询，不会出现旧数据，也不会因旧索引文件残留导致错误。

测试用例四：删除不存在的表

```
drop table not_exist_table;
```

预期结果：

系统返回失败，说明能够正确识别表不存在的异常情况。

3. 实现思路的简要分析流程

MiniOB 中一条 SQL 语句的执行大致经过如下流程：

SQL 文本

- Parser 语法解析
- ParsedSqlNode
- Stmt 语句对象
- Executor 执行器

- Db/Table 存储层
- 文件与内存资源操作

因此，实现 drop table 功能时，整体思路是沿着 MiniOB 原有的 SQL 执行链路补全对应模块。具体来说，需要让系统能够将 drop table 表名；解析为删除表语句对象，再由执行器调用数据库层的删除表接口，最终完成表元数据文件、数据文件、索引文件以及内存对象的释放。

3.1 解析层：识别 drop table 语句

当前 MiniOB 的语法解析层已经能够识别类似如下 SQL：

```
drop table t;
```

其语义是删除表 t。Parser 解析后会生成对应的 DropTableSqlNode，其中保存待删除的表名。由于当前版本中语法结构已经基本存在，因此本次实现重点不在 parser，而在后续的 Stmt、Executor 和存储层。

3.2 Stmt 层：增加 DropTableStmt

Stmt 层的作用是把 Parser 生成的语法树节点转换成 MiniOB 内部更方便执行的语句对象。因此，为 drop table 新增 DropTableStmt 类，用于保存待删除表名。

关键代码如下：

```
class DropTableStmt : public Stmt
{
public:
    explicit DropTableStmt(const string &table_name) :
        table_name_(table_name) {}
    virtual ~DropTableStmt() = default;

    StmtType type() const override { return StmtType::DROP_TABLE; }

    const string &table_name() const { return table_name_; }

    static RC create(Db *db, const DropTableSqlNode &drop_table, Stmt
*&stmt);

private:
    string table_name_;
};
```

在 DropTableStmt::create 中，对数据库对象和表名进行基本检查，然后创建对应的 DropTableStmt 对象：

```
RC DropTableStmt::create(Db *db, const DropTableSqlNode &drop_table, Stmt
*&stmt)
{
    stmt = nullptr;

    const char *table_name = drop_table.relation_name.c_str();
    if (db == nullptr || is_blank(table_name)) {
        return RC::INVALID_ARGUMENT;
    }

    stmt = new DropTableStmt(drop_table.relation_name);
    return RC::SUCCESS;
}
```

随后，在 Stmt::create_stmt 中增加对 SCF_DROP_TABLE 的处理：

```
case SCF_DROP_TABLE: {
    return DropTableStmt::create(db, sql_node.drop_table, stmt);
}
```

这样，系统就可以把解析后的 drop table 语法节点转换为可执行的语句对象。

3.3 Executor 层：增加 DropTableExecutor

Executor 层负责真正执行语句。对于 drop table，执行器本身不直接删除文件，而是取得当前数据库对象，并调用数据库层的 drop_table 接口。

关键代码如下：

```
RC DropTableExecutor::execute(SQLStageEvent *sql_event)
{
    Stmt *stmt = sql_event->stmt();
    ASSERT(stmt->type() == StmtType::DROP_TABLE,
        "drop table executor can not run this command");

    DropTableStmt *drop_table_stmt = static_cast<DropTableStmt *>(stmt);
    Session *session = sql_event->session_event()->session();

    return
    session->get_current_db()->drop_table(drop_table_stmt->table_name().c_str
());
}
```

然后在 CommandExecutor 中增加对应分支：

```
case StmtType::DROP_TABLE: {
    DropTableExecutor executor;
    rc = executor.execute(sql_event);
} break;
```

这样，当系统识别到当前语句类型为 DROP_TABLE 时，就会调用 DropTableExecutor 完成执行。

3.4 Db 层：从数据库中移除表对象

Db 层负责管理当前数据库中已经打开的表对象。MiniOB 中已经打开的表会保存在 `opened_tables_` 中，因此删除表时首先需要检查表是否存在，然后调用 `Table::drop` 删除表的底层资源，最后从 `opened_tables_` 中移除该表对象。

关键代码如下：

```
RC Db::drop_table(const char *table_name)
{
    if (common::is_blank(table_name)) {
        return RC::INVALID_ARGUMENT;
    }

    auto it = opened_tables_.find(string(table_name));
    if (it == opened_tables_.end()) {
        return RC::SCHEMA_TABLE_NOT_EXIST;
    }

    Table *table = it->second;

    RC rc = table->drop(path_.c_str());
    if (rc != RC::SUCCESS) {
        return rc;
    }

    opened_tables_.erase(it);
    delete table;

    return RC::SUCCESS;
}
```

这一部分解决的是数据库对象层面的删除问题：如果只删除磁盘文件，而不从 `opened_tables_` 中移除表对象，系统仍然可能认为该表存在，后续重建同名表或查询表时会出现异常。

3.5 Table 层：删除表相关文件并释放内存资源

Table 层负责具体删除表相关资源。删除表时，不能只删除 `.table` 元数据文件和 `.data` 数据文件，还要处理索引文件和内存中的表引擎、索引、缓冲池等资源。

实现时，先调用 `sync()`，确保在删除前脏页已经刷盘；然后释放 `engine_`，使表数据文件、索引对象以及对应的 `buffer pool` 资源被关闭；最后删除表的元数据文件、数据文件和索引文件。

关键代码如下：

```
RC Table::drop(const char *base_dir)
{
```

```

if (common::is_blank(base_dir)) {
    return RC::INVALID_ARGUMENT;
}

RC rc = sync();
if (rc != RC::SUCCESS) {
    return rc;
}

const string table_name = name();

// 释放表引擎，关闭数据文件、索引对象和 buffer pool
engine_.reset();

if (lob_handler_ != nullptr) {
    delete lob_handler_;
    lob_handler_ = nullptr;
}

auto remove_file = [](const string &file) -> RC {
    if (::unlink(file.c_str()) != 0) {
        if (errno == ENOENT) {
            return RC::SUCCESS;
        }
        return RC::FILE_REMOVE;
    }
    return RC::SUCCESS;
};

string meta_file = table_meta_file(base_dir, table_name.c_str());
rc = remove_file(meta_file);
if (rc != RC::SUCCESS) {
    return rc;
}

string data_file = table_data_file(base_dir, table_name.c_str());
rc = remove_file(data_file);
if (rc != RC::SUCCESS) {
    return rc;
}

const int index_num = table_meta_.index_num();
for (int i = 0; i < index_num; i++) {
    const IndexMeta *index_meta = table_meta_.index(i);
    if (index_meta == nullptr) {
        continue;
    }

    string index_file = table_index_file(base_dir, table_name.c_str(),
index_meta->name());
    rc = remove_file(index_file);
    if (rc != RC::SUCCESS) {
        return rc;
    }
}

return RC::SUCCESS;
}

```

drop table 的本质不仅是删除表名对应的元数据，还要完整清理该表关联的所有资源。如果没有释放内存中的表引擎和缓冲池，即使磁盘文件被删除，之后重新创建同名表时仍可能读到旧缓存中的数据。因此，本次实现中特别加入了 `engine_.reset()`，确保表相关的内存状态被释放。

3.6 异常情况处理

实现中还考虑了表不存在的情况。当用户执行：

```
drop table not_exist_table;
```

如果 `opened_tables_` 中找不到对应表名，则返回：

```
RC:::SCHEMA_TABLE_NOT_EXIST
```

这样系统会向客户端返回执行失败，避免错误地删除不存在的资源。

4. 测试结果

```
miniob > insert into t values(1, 'aaa');
SUCCESS
miniob > insert into t values(2, 'bbb');
SUCCESS
miniob > select * from t;
age | name
1 | aaa
2 | bbb
miniob > drop table t;
SUCCESS
miniob > select * from t;
FAILURE
```

```
miniob > create table t3(id int, name char(20));
SUCCESS
miniob > insert into t3 values(1, 'aaa');
SUCCESS
miniob > insert into t3 values(2, 'bbb');
SUCCESS
miniob > create index idx_t3_id on t3(id);
SUCCESS
miniob > select * from t3;
id | name
1 | aaa
2 | bbb
miniob > drop table t3;
SUCCESS
miniob > create table t3(id int, name char(20));
SUCCESS
miniob > select * from t3;
id | name
miniob > drop table t3;
SUCCESS
```

```
miniob > create table t2(id int, name char(20));
SUCCESS
miniob > insert into t2 values(1, 'old');
SUCCESS
miniob > select * from t2;
id | name
1 | old
miniob > drop table t2;
SUCCESS
miniob > create table t2(id int, name char(20));
SUCCESS
miniob > select * from t2;
id | name
miniob > drop table t2;
SUCCESS
```

```
miniob > drop table not_exist_table;
FAILURE
```

均符合预期

二. date

1. 赛题基本信息

赛题描述:

本题要求在 MiniOB 现有功能基础上实现 DATE 类型字段。当前 MiniOB 已经支持 INT、CHAR、FLOAT 等基本类型，本题需要在此基础上增加 DATE 类型，使系统能够支持如下 SQL：

```
CREATE TABLE t(id INT, birthday DATE);
INSERT INTO t VALUES(1, '2022-10-10');
```

实现时需要考虑以下内容：

1. 语法解析层能够识别 DATE 类型；
2. 类型系统中增加 DATE 类型；
3. 插入数据时能够把字符串形式的日期转换为内部存储格式；
4. 查询输出时能够把内部日期值重新格式化为 YYYY-MM-DD；
5. 查询条件中能够正确比较日期大小；
6. 需要检查非法日期输入，例如月份非法、日期非法、闰年错误等；
7. 日期范围不能依赖 time_t，因为测试数据可能大于 2038 年 2 月，也可能小于 1970 年 1 月 1 日。

2. 测试用例示例

测试用例一：基础 DATE 建表、插入和查询

```
drop table t;
create table t(id int, birthday date);
insert into t values(1, '2022-10-10');
insert into t values(2, '2021-1-2');
select * from t;
```

预期结果：

```
id | birthday
1 | 2022-10-10
2 | 2021-01-02
```

系统能够识别 DATE 类型字段，并且能够把字符串形式的日期插入表中。对于 '2021-1-2' 这种输入，输出时应格式化为标准形式 2021-01-02。

测试用例二：闰年合法性检查

```
insert into t values(3, '2024-02-29');
select * from t where birthday = '2024-02-29';
```

预期结果：

插入和查询均成功，因为 2024 年是闰年，2024-02-29 是合法日期。

测试用例三：非法日期检查

```
insert into t values(4, '2022-02-29');
insert into t values(5, '1900-02-29');
insert into t values(6, '2022-13-01');
insert into t values(7, '2022-00-01');
insert into t values(8, '2022-04-31');
insert into t values(9, '2022-01-00');
```

预期结果：

以上语句均返回 FAILURE。2022-02-29 非闰年非法；1900-02-29 因 1900 年不是闰年非法；2022-13-01 月份非法；2022-04-31 因 4 月没有 31 日非法。

测试用例四：测试 1970/2038 边界之外的日期

```
insert into t values(10, '1969-12-31');
insert into t values(11, '2040-01-01');
insert into t values(12, '9999-12-31');
select * from t;
```

预期结果：

以上日期均可成功插入并正常显示。本题不能使用依赖系统时间戳的存储方式，否则可能受到 1970 年和 2038 年边界限制。因此实现时采用 YYYYMMDD 整数编码存储日期。

测试用例五：日期比较查询

```
select * from t where birthday > '2022-01-01';
select * from t where birthday < '1970-01-01';
select * from t where birthday >= '2040-01-01';
select * from t where birthday = '2022-10-10';
```

预期结果：

系统能够按照日期先后顺序正确筛选结果。

测试用例六：查询条件中的非法日期检查

```
select * from t where birthday = '2022-02-29';
select * from t where birthday > '2022-13-01';
```

预期结果：

FAILURE

FAILURE

不仅插入时需要检查日期合法性，where 条件中的日期常量也需要被转换并检查。如果查询条件中出现非法日期，应直接返回 FAILURE。

3. 实现思路的简要分析流程

MiniOB 中一条 SQL 的执行大致经过如下流程：

SQL 文本

- 词法/语法解析 Parser
- ParsedSqlNode
- Stmt
- Logical/Physical Plan
- Executor
- Table/Record/Index 存储层

本题实现 DATE 类型，需要沿着 MiniOB 的 SQL 解析、类型系统、值存储、类型转换、表达式比较和结果输出等模块进行扩展。整体思路是：在语法层增加 DATE 关键字，在类型枚举中增加 DATES，在 Value 和 DataType 中增加日期类型处理，并使用 4 字节整数 YYYYMMDD 作为内部存储格式。这样可以避免 time_t 在 1970 年和 2038 年附近的限制，同时整数大小顺序也与日期先后顺序一致。

3.1 增加 DATE 类型枚举

首先在 MiniOB 的属性类型枚举中增加 DATES 类型，使表字段可以声明为日期类型。

关键代码如下：

```
enum class AttrType
{
    UNDEFINED,
    CHARS,
    INTS,
    FLOATS,
    VECTORS,
    BOOLEANS,
    DATES,
    MAXTYPE,
};
```

同时在类型名称数组中增加 "dates":

```
const char *ATTR_TYPE_NAME[] = {
    "undefined",
    "chars",
    "ints",
    "floats",
    "vectors",
    "booleans",
    "dates"
};
```

这一步的作用是让 MiniOB 的元数据系统能够识别和保存 DATE 字段类型。数据库的数据字典需要保存关系名、属性名、属性类型等元数据，课件中也将这些信息列为数据字典存储内容的一部分。

3.2 修改 Parser，使建表语句支持 DATE

为了让系统识别如下语句:

```
create table t(id int, birthday date);
```

需要修改词法分析和语法分析部分。在 lex_sql.1 中增加 DATE 关键字:

```
DATE      RETURN_TOKEN(DATE_T);
```

该规则需要放在普通标识符规则之前，否则 DATE 会被识别成普通 ID。

然后在 yacc_sql.y 中增加 token:

```
%token DATE_T
```

并在字段类型规则中加入:

```
type:
    INT_T      { $$ = static_cast<int>(AttrType::INTS); }
  | STRING_T   { $$ = static_cast<int>(AttrType::CHARS); }
  | FLOAT_T    { $$ = static_cast<int>(AttrType::FLOATS); }
  | VECTOR_T   { $$ = static_cast<int>(AttrType::VECTORS); }
```

```
| DATE_T    { $$ = static_cast<int>(AttrType::DATES); }
;
```

这样，建表语句中的 birthday DATE 就能被解析为 DATES 类型字段。

3.3 在 Value 中增加 DATE 值的存取接口

MiniOB 使用 Value 表示 SQL 中的常量值和记录中的字段值。因此需要让 Value 支持保存日期值。本次实现中，日期内部采用 int 存储，格式为：YYYYMMDD

例如：

```
2022-10-10 → 20221010
2024-02-29 → 20240229
1969-12-31 → 19691231
```

在 Value 中增加接口：

```
void set_date(int val);
int get_date() const;
```

关键实现如下：

```
void Value::set_date(int val)
{
    reset();
    attr_type_ = AttrType::DATES;
    value_.int_value_ = val;
    length_ = sizeof(val);
}
int Value::get_date() const
{
    switch (attr_type_) {
        case AttrType::DATES:
        case AttrType::INTS:
            return value_.int_value_;

        default:
            return 0;
    }
}
```

由于 DATE 内部采用 4 字节整数存储，因此它在记录中的存储方式与 INT 类似，便于和 MiniOB 原有定长字段存储结构兼容。

3.4 增加 DateType 类型类

MiniOB 的类型系统通过不同的 DataType 子类处理不同数据类型的比较、转换和输出。因此新增 DateType 类，用于处理日期类型。

DateType 的主要职责包括：

1. 解析字符串日期；
2. 检查日期合法性；
3. 将日期转为内部整数；
4. 比较两个日期值；
5. 将内部日期值格式化为字符串输出。

关键代码如下：

```
class DateType : public DataType
{
public:
    DateType() : DataType(AttrType::DATES) {}
    virtual ~DateType() = default;

    int compare(const Value &left, const Value &right) const override;
    RC cast_to(const Value &val, AttrType type, Value &result) const
    override;
    RC set_value_from_str(Value &val, const string &data) const override;
    RC to_string(const Value &val, string &result) const override;

    static bool parse_date(const string &s, int &date_value);

private:
    static bool is_leap_year(int year);
};
```

日期合法性判断中需要考虑闰年：

```
bool DateType::is_leap_year(int year)
{
    return (year % 400 == 0) || (year % 4 == 0 && year % 100 != 0);
}
```

日期解析时，对年份、月份和天数进行检查。若合法，则编码为 YYYYMMDD：

```
date_value = year * 10000 + month * 100 + day;
```

非法日期则返回错误：

```
if (!parse_date(data, date_value)) {
    return RC::SCHEMA_FIELD_TYPE_MISMATCH;
}
```

这样可以保证：

2024-02-29 合法

2022-02-29 非法

2000-02-29 合法
1900-02-29 非法
2022-13-01 非法
2022-04-31 非法

3.5 注册 DataType

新增 DataType 后, 需要在 DataType::type_instances_ 中注册该类型:

```
array<unique_ptr<DataType>, static_cast<int>(AttrType::MAXTYPE)>
DataType::type_instances_ = {
    make_unique<DataType>(AttrType::UNDEFINED),
    make_unique<CharType>(),
    make_unique<IntegerType>(),
    make_unique<FloatType>(),
    make_unique<VectorType>(),
    make_unique<DataType>(AttrType::BOOLEANS),
    make_unique<DateType>(),
};
```

这里注册顺序必须与 AttrType 枚举顺序一致。否则通过 AttrType::DATES 获取类型实例时, 会取到错误类型。

3.6 实现字符串到 DATE 的类型转换

SQL 插入日期时, 日期通常以字符串形式出现:

```
insert into t values(1, '2022-10-10');
```

Parser 会先把 '2022-10-10' 解析为 CHARS 类型常量。而表字段类型是 DATES, 因此插入时需要进行类型转换。

在 CharType::cast_to 中增加 CHARS → DATES 转换:

```
RC CharType::cast_to(const Value &val, AttrType type, Value &result)
const
{
    switch (type) {
        case AttrType::DATES: {
            return DataType::type_instance(AttrType::DATES)
                ->set_value_from_str(result, val.get_string());
        }

        case AttrType::CHARS: {
            result.set_string(val.get_string().c_str());
            return RC::SUCCESS;
        }

        default:
            return RC::SCHEMA_FIELD_TYPE_MISMATCH;
    }
}
```

```
}
}
```

这样，插入 DATE 字段时，字符串常量会被解析、检查并转换为内部整数格式。如果日期非法，则转换失败，最终 SQL 返回 FAILURE。

3.7 实现 DATE 的输出格式化

DATE 内部使用整数存储，但用户查询时应看到标准日期格式。因此在 `DateType::to_string` 中将内部整数重新格式化为 YYYY-MM-DD。

关键代码如下：

```
RC DateType::to_string(const Value &val, string &result) const
{
    int date_value = val.get_date();

    int year = date_value / 10000;
    int month = (date_value / 100) % 100;
    int day = date_value % 100;

    char buffer[16] = {0};
    snprintf(buffer, sizeof(buffer), "%04d-%02d-%02d", year, month, day);
    result = buffer;

    return RC::SUCCESS;
}
```

因此，即使输入：`insert into t values(2, '2021-1-2');`

查询输出也会变成：2021-01-02

3.8 实现 DATE 比较

日期内部编码为 YYYYMMDD，因此可以直接用整数大小比较日期先后：

```
int DateType::compare(const Value &left, const Value &right) const
{
    int left_date = left.get_date();
    int right_date = right.get_date();

    if (left_date < right_date) {
        return -1;
    }
    if (left_date > right_date) {
        return 1;
    }
    return 0;
}
```


由于：19691231 < 20221010 < 20400101，所以整数比较结果与日期先后顺序一致。这样可以支持：

```
select * from t where birthday > '2022-01-01';
select * from t where birthday < '1970-01-01';
select * from t where birthday >= '2040-01-01';
```

4. 测试结果

```
miniob > create table t(id int, birthday date);
SUCCESS
miniob > insert into t values(1, '2022-10-10');
SUCCESS
miniob > insert into t values(2, '2021-1-2');
SUCCESS
miniob > select * from t;
id | birthday
1 | 2022-10-10
2 | 2021-01-02
```

```
miniob > insert into t values(3, '2024-02-29');
SUCCESS
miniob > select * from t where birthday = '2024-02-29';
id | birthday
3 | 2024-02-29
```

```
miniob > insert into t values(4, '2022-02-29');
FAILURE
miniob > insert into t values(5, '1900-02-29');
FAILURE
miniob > insert into t values(6, '2022-13-01');
FAILURE
miniob > insert into t values(7, '2022-00-01');
FAILURE
miniob > insert into t values(8, '2022-04-31');
FAILURE
miniob > insert into t values(9, '2022-01-00');
FAILURE
```

```

miniob > insert into t values(10, '1969-12-31');
SUCCESS
miniob > insert into t values(11, '2040-01-01');
SUCCESS
miniob > insert into t values(12, '9999-12-31');
SUCCESS
miniob > select * from t;
id | birthday
1 | 2022-10-10
2 | 2021-01-02
3 | 2024-02-29
10 | 1969-12-31
11 | 2040-01-01
12 | 9999-12-31

miniob > select * from t where birthday > '2022-01-01';
id | birthday
1 | 2022-10-10
3 | 2024-02-29
11 | 2040-01-01
12 | 9999-12-31
miniob > select * from t where birthday < '1970-01-01';
id | birthday
10 | 1969-12-31
miniob > select * from t where birthday >= '2040-01-01';
id | birthday
11 | 2040-01-01
12 | 9999-12-31
miniob > select * from t where birthday = '2022-10-10';
id | birthday
1 | 2022-10-10

```

三. like

1. 赛题基本信息

赛题描述:

本题要求在 MiniOB 中实现 SQL 的 LIKE 字符串匹配功能，使系统能够在 WHERE 条件中使用 LIKE 或 NOT LIKE 对字符型字段进行模式匹配查询。

SQL 中的 LIKE 是用于字符型数据的字符串匹配运算符。，% 可以匹配任意子串，_ 可以匹配任意一个字符。同时，当待匹配的字符中本身包含百分号时，需要使用转义形式，例如 like 'Watson\%'。

本题实现的主要目标包括:

1. 支持 LIKE 关键字的词法和语法解析;
2. 支持 NOT LIKE 形式;
3. 支持 % 匹配任意长度字符串;
4. 支持 _ 匹配任意单个字符;

5. 支持通过反斜杠转义 % 和 _;
6. 在查询执行阶段根据匹配结果筛选元组。

2. 测试用例示例

测试用例一：准备测试数据

```
drop table t;
create table t(id int, name char(20));
insert into t values(1, 'Alice');
insert into t values(2, 'Bob');
insert into t values(3, 'Bobby');
insert into t values(4, 'Rob');
insert into t values(5, 'Tom');
insert into t values(6, 'A_100');
insert into t values(7, 'A%200');
select * from t;
```

预期结果：

表 t 创建成功，测试数据插入成功，查询结果中包含上述 7 条记录。

测试用例二：% 匹配任意子串

```
select * from t where name like 'Bo%';
```

预期结果：

id	name
2	Bob
3	Bobby

Bo% 表示以 Bo 开头的任意字符串，因此 Bob 和 Bobby 均满足条件。

```
select * from t where name like '%ob%';
```

预期结果：

id	name
2	Bob
3	Bobby
4	Rob

%ob% 表示字符串中任意位置包含子串 ob。

测试用例三：_ 匹配单个字符

```
select * from t where name like '_ob';
```

预期结果：

id	name
2	Bob
4	Rob

_ob 只能匹配长度为 3，且后两个字符为 ob 的字符串，因此 Bob 和 Rob 满足条件，Bobby 不满足条件。

测试用例四：普通字符串完全匹配

```
select * from t where name like 'Alice';
```

预期结果：

id	name
1	Alice

当模式串中不包含 % 或 _ 时，LIKE 的效果等价于普通字符串完全匹配。

测试用例五：NOT LIKE

```
select * from t where name not like 'Bo%';
```

预期结果：

查询结果中不包含 Bob 和 Bobby，其余记录返回。

测试用例六：转义 % 和 _

```
select * from t where name like 'A\_%';
```

预期结果：

id	name
6	A_100

A_% 中的 _ 表示普通下划线字符，最后的 % 表示任意后缀，因此匹配 A_100。

```
select * from t where name like 'A\%%';
```

预期结果：

```
id | name
7 | A%200
```

A\%% 中的 \% 表示普通百分号字符，最后的 % 表示任意后缀，因此匹配 A%200。

3. 实现思路的简要分析流程

本题的实现不需要修改底层存储格式，而是主要沿着 MiniOB 的 SQL 解析和表达式求值链路进行扩展。实现重点包括：在 Parser 中识别 LIKE 和 NOT LIKE，在比较运算符中增加对应类型，并在表达式求值阶段实现字符串模式匹配。

3.1 增加 LIKE 和 NOT LIKE 比较运算符

MiniOB 原有的 CompOp 枚举中已经包含 =, <=, <>, <, >=, > 等普通比较运算符。本题需要在此基础上增加 LIKE_OP 和 NOT_LIKE_OP，用于表示 LIKE 和 NOT LIKE 两种字符串匹配条件。

关键代码如下：

```
enum CompOp
{
    EQUAL_TO,      ///< "="
    LESS_EQUAL,    ///< "<="
    NOT_EQUAL,     ///< "<>"
    LESS_THAN,     ///< "<"
    GREAT_EQUAL,   ///< ">="
    GREAT_THAN,    ///< ">"
    LIKE_OP,       ///< "LIKE"
    NOT_LIKE_OP,   ///< "NOT LIKE"
    NO_OP
};
```

将新增运算符放在 NO_OP 之前，可以使原有合法性判断继续生效，避免 LIKE_OP 和 NOT_LIKE_OP 被误判为非法比较运算符。

3.2 修改词法分析，识别 LIKE 和 NOT

MiniOB 的 SQL 解析模块是接收到用户请求后正式处理 SQL 的第一步，会将用户输入转换成内部语法树。其解析模块位于 `src/observer/sql/parser`，其中 `lex_sql.l` 是词法解析代码，`yacc_sql.y` 是语法解析代码，`parse_defs.h` 中包含语法树相关数据结构。词法解析会把输入 SQL 解析成 token，语法分析再根据 token 生成内部结构。

因此，需要在 `lex_sql.l` 中增加 LIKE 和 NOT 关键字的 token 规则：

```
LIKE          RETURN_TOKEN(LIKE);
NOT           RETURN_TOKEN(NOT);
```

该规则需要放在普通标识符 ID 的匹配规则之前，否则 LIKE 和 NOT 可能会被识别为普通字段名或表名。

3.3 修改语法分析，使 LIKE 进入条件表达式

在 `yacc_sql.y` 中增加 token 声明：

```
%token LIKE NOT
```

随后在比较运算符规则 `comp_op` 中增加 LIKE 和 NOT LIKE：

```
comp_op:
    EQ { $$ = EQUAL_TO; }
  | LT { $$ = LESS_THAN; }
  | GT { $$ = GREAT_THAN; }
  | LE { $$ = LESS_EQUAL; }
  | GE { $$ = GREAT_EQUAL; }
  | NE { $$ = NOT_EQUAL; }
  | LIKE { $$ = LIKE_OP; }
  | NOT LIKE { $$ = NOT_LIKE_OP; }
  ;
```

这样，用户输入：`select * from t where name like 'Bo%'`；时，Parser 可以将 like 识别为比较运算符，并将其保存到条件节点 `ConditionSqlNode` 中。由于 MiniOB 原有条件表达式已经支持 `rel_attr comp_op value` 等形式，因此只要扩展 `comp_op`，LIKE 便可以自然复用原有的条件解析框架。

3.4 实现 LIKE 模式匹配函数

LIKE 的核心是模式串匹配。与普通字符串比较不同，LIKE 中的 % 和 _ 具有特殊含义：

% 匹配任意长度字符串，包括空串

_ 匹配任意一个字符

\% 匹配普通百分号

_ 匹配普通下划线

本次实现中，先将模式串拆分为 token，再使用动态规划判断文本串是否匹配模式串。
关键结构如下：

```
struct LikeToken
{
    enum class Type
    {
        LITERAL,
        ANY_ONE,    // _
        ANY_MANY    // %
    };

    Type type;
    char ch;
};
```

其中：

LITERAL 表示普通字符：

ANY_ONE 表示通配符 _：

ANY_MANY 表示通配符 %。

模式串解析函数如下：

```
static vector<LikeToken> tokenize_like_pattern(const string &pattern)
{
    vector<LikeToken> tokens;

    for (size_t i = 0; i < pattern.size(); i++) {
        char ch = pattern[i];

        if (ch == '\\' && i + 1 < pattern.size()) {
            tokens.push_back({LikeToken::Type::LITERAL, pattern[i + 1]});
            i++;
        } else if (ch == '%') {
            tokens.push_back({LikeToken::Type::ANY_MANY, 0});
        } else if (ch == '_') {
            tokens.push_back({LikeToken::Type::ANY_ONE, 0});
        } else {
            tokens.push_back({LikeToken::Type::LITERAL, ch});
        }
    }

    return tokens;
}
```

该函数可以正确区分普通字符、通配符和转义字符。

3.5 使用动态规划完成字符串匹配

定义：

$dp[i][j]$ 表示 $text$ 的前 i 个字符能否匹配 $pattern$ 的前 j 个 $token$ 。

转移规则如下：

1. 若当前 $token$ 是 $\%$ ，则它可以匹配空串，也可以匹配至少一个字符；
2. 若当前 $token$ 是 $_$ ，则它匹配任意单个字符；
3. 若当前 $token$ 是普通字符，则要求当前字符相等。

关键代码如下：

```
static bool like_match(const string &text, const string &pattern)
{
    vector<LikeToken> tokens = tokenize_like_pattern(pattern);

    const int m = static_cast<int>(text.size());
    const int n = static_cast<int>(tokens.size());

    vector<vector<uint8_t>> dp(m + 1, vector<uint8_t>(n + 1, 0));
    dp[0][0] = 1;

    for (int j = 1; j <= n; j++) {
        if (tokens[j - 1].type == LikeToken::Type::ANY_MANY) {
            dp[0][j] = dp[0][j - 1];
        }
    }

    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            const LikeToken &token = tokens[j - 1];

            if (token.type == LikeToken::Type::ANY_MANY) {
                dp[i][j] = dp[i][j - 1] || dp[i - 1][j];
            } else if (token.type == LikeToken::Type::ANY_ONE) {
                dp[i][j] = dp[i - 1][j - 1];
            } else {
                dp[i][j] = dp[i - 1][j - 1] && text[i - 1] == token.ch;
            }
        }
    }

    return dp[m][n] != 0;
}
```

这种实现方式较为稳妥，能够统一处理前缀匹配、后缀匹配、子串匹配、单字符匹配和全字符串匹配等情况。

3.6 在比较表达式中处理 LIKE 和 NOT LIKE

MiniOB 中 WHERE 条件最终会进入比较表达式求值逻辑。原有的 `ComparisonExpr::compare_value` 主要处理普通大小比较，例如 `=`, `<`, `>` 等。本题在该函数中增加对 `LIKE_OP` 和 `NOT_LIKE_OP` 的专门处理。

关键代码如下：

```
RC ComparisonExpr::compare_value(const Value &left, const Value &right,
bool &result) const
{
    RC rc = RC::SUCCESS;
    .....
    result = false;

    if (comp_ == LIKE_OP || comp_ == NOT_LIKE_OP) {
        if (left.attr_type() != AttrType::CHARS || right.attr_type() !=
AttrType::CHARS) {
            return RC::SCHEMA_FIELD_TYPE_MISMATCH;
        }

        bool matched = like_match(left.get_string(), right.get_string());
        result = (comp_ == LIKE_OP) ? matched : !matched;
        return RC::SUCCESS;
    }
    .....
}
```

3.7 兼容向量化求值路径

MiniOB 查询执行过程中可能存在普通逐元组求值路径和向量化求值路径。为了避免 `LIKE` 在向量化路径中被误当作普通比较运算符处理，本次实现还在 `ComparisonExpr::eval` (`Chunk &chunk`, `vector<uint8_t> &select`) 中加入对 `LIKE_OP` 和 `NOT_LIKE_OP` 的专门判断。

核心逻辑是：逐行取出左右两侧的 `Value`，调用 `compare_value` 进行匹配，并根据结果更新 `select` 数组。

关键代码如下：

```
if (comp_ == LIKE_OP || comp_ == NOT_LIKE_OP) {
    if (left_column.attr_type() != AttrType::CHARS ||
        right_column.attr_type() != AttrType::CHARS) {
        return RC::SCHEMA_FIELD_TYPE_MISMATCH;
    }

    int rows = left_column.column_type() == Column::Type::CONSTANT_COLUMN ?
        right_column.count() : left_column.count();

    for (int i = 0; i < rows; ++i) {
        Value left_val = left_column.get_value(i);
```

```

Value right_val = right_column.get_value(i);

bool result = false;
rc = compare_value(left_val, right_val, result);
if (rc != RC::SUCCESS) {
    return rc;
}

select[i] &= result ? 1 : 0;
}

return RC::SUCCESS;
}

```

这样无论查询使用普通表达式求值路径还是向量化表达式求值路径，都可以正确支持 LIKE。

4. 测试结果

```

miniob > select * from t;
id | name
1 | Alice
2 | Bob
3 | Bobby
4 | Rob
5 | Tom
6 | A_100
7 | A%200
miniob > select * from t where name like 'Bo%';
id | name
2 | Bob
3 | Bobby
miniob > select * from t where name like '%ob%';
id | name
2 | Bob
3 | Bobby
4 | Rob
miniob > select * from t where name like '_ob';
id | name
2 | Bob
4 | Rob
miniob > select * from t where name like 'Alice';
id | name
1 | Alice
miniob > select * from t where name not like 'Bo%';
id | name
1 | Alice
4 | Rob
5 | Tom
6 | A_100
7 | A%200
miniob > select * from t where name like 'A\_%';
id | name
6 | A_100
miniob > select * from t where name like 'A\%%';
id | name
7 | A%200

```

出现的问题：

实现过程中发现一个问题：插入非法日期能够返回 FAILURE，但查询条件中的非法日期最初没有被检查。例如：

```
select * from t where birthday > '2022-13-01';
```

原本可能被系统转换为：

```
CAST(birthday AS CHARS) > '2022-13-01'
```

这样右侧 '2022-13-01' 只是普通字符串，不会被当作 DATE 解析，因此不会报错。

```
miniob > select * from t where birthday = '2022-02-29';
id | birthday
miniob > select * from t where birthday > '2022-13-01';
id | birthday
3 | 2024-02-29
11 | 2040-01-01
12 | 9999-12-31
```

解决方案：

为了解决这个问题，需要修改隐式转换代价，使系统在 DATE 字段与字符串常量比较时，选择：

```
birthday > CAST('2022-13-01' AS DATE)
```

而不是把 DATE 字段转成字符串。

在 CharType::cast_cost 中允许 CHARS → DATES：

```
int CharType::cast_cost(AttrType type)
{
    if (type == AttrType::CHARS) {
        return 0;
    }

    if (type == AttrType::DATES) {
        return 1;
    }

    return INT32_MAX;
}
```

同时在 DateType::cast_cost 中禁止 DATES → CHARS 作为隐式转换：

```
int cast_cost(AttrType type) override
{
    if (type == AttrType::DATES) {
        return 0;
    }

    return INT32_MAX;
}
```

这样对于：

```
select * from t where birthday > '2022-13-01';
```

系统会尝试把右侧字符串常量转换为 DATE。由于 2022-13-01 是非法日期，转换失败，最终 SQL 返回 FAILURE。

```
miniob > select * from t where birthday = '2022-02-29';
FAILURE
miniob > select * from t where birthday > '2022-13-01';
FAILURE
```